

Project GEWN

AI-Native Desktop Copilot Platform

A futuristic floating AI assistant designed to redefine human-computer interaction — combining conversational AI, desktop automation, multimodal screen understanding, semantic memory, and autonomous agent orchestration into a unified intelligent operating companion.

Table of Contents

- [Overview](#)
 - [Why GEWN](#)
 - [Core Capabilities](#)
 - [Architecture](#)
 - [Tech Stack](#)
 - [AI Behavior & Safety](#)
 - [Security Model](#)
 - [User Interaction Model](#)
 - [Roadmap](#)
 - [Project Structure](#)
 - [Documentation](#)
 - [Development Status](#)
-

Overview

Project GEWN is a next-generation AI desktop assistant that lives directly on top of the user's screen as a persistent, intelligent overlay. Rather than functioning as an isolated chatbot, GEWN acts as an operating layer — one that understands context, controls the computer, executes complex workflows, remembers the user's preferences and history, and provides proactive assistance across every part of the desktop environment.

The vision is a computing experience where the AI doesn't wait to be asked — it understands what you're working on, anticipates what you need, and acts with both intelligence and restraint.

Inspired by the AI companions of science fiction — Jarvis, J.F.K.I., Spider-Verse-style operating systems — but built for real-world, everyday computing workflows.

Why GEWN

Modern computers still demand manual operation for nearly everything. Users switch between applications constantly, repeat the same setup routines every day, search for files they created weeks ago, copy information between tools that have no awareness of each other, and lose cognitive energy navigating the OS instead of doing the actual work.

Existing AI assistants address this poorly. They operate inside isolated chat windows, lack awareness of what's on screen, cannot meaningfully interact with the operating system, and fail to act on the user's behalf in any autonomous capacity.

GEWN is designed to close this gap — an AI system that:

- **Understands context in real time** — what application is open, what's on screen, what you're working toward
 - **Acts, not just answers** — controls applications, manages files, automates workflows, browses the web
 - **Remembers** — maintains persistent semantic memory across sessions and evolves with usage
 - **Stays out of the way** — proactive but non-intrusive; assistive without being disruptive
-

Core Capabilities

Conversational AI

Natural language interaction with streaming responses, full conversation history, follow-up understanding, Markdown and code rendering, and context injected from the current session and long-term memory. Powered by frontier LLMs with multi-provider support.

Floating Desktop Overlay

An always-on-top transparent glassmorphism UI that floats above all applications. Draggable, resizable, minimizable to a small icon. Accessible instantly via keyboard shortcut. Built in Electron + React with Framer Motion animations.

Voice Interaction

Full voice pipeline: wake word detection ("Hey GEWN"), high-accuracy speech-to-text, emotion-aware text-to-speech, and a real-time continuous voice mode with interruption handling. Designed for natural conversation with sub-1.5-second perceived latency. Local and cloud STT options available.

Desktop Automation

Direct OS-level interaction — creating, moving, organizing, and deleting files; launching and controlling applications; managing windows; executing terminal commands with safety validation; clipboard management. All automation is risk-classified and requires tiered user approval for consequential actions.

Browser Automation

Playwright-powered browser agent capable of navigating websites, extracting and summarizing web content, filling and submitting forms (with user approval), downloading files, and executing multi-tab research workflows.

Screen Awareness & Vision

On-demand or passive screen capture, OCR extraction via PaddleOCR/Tesseract, and multimodal LLM analysis that understands UI elements, coding errors, document layouts, and visual context. Includes visual overlay guidance — transparent annotations and step-by-step interactive overlays rendered directly on screen.

AI Memory System

Three-layer memory architecture: short-term session memory (Redis), long-term semantic memory (PostgreSQL + ChromaDB), and workflow memory that learns habits and recurring patterns. Users can view, edit, and delete all stored memories at any time. A Second Brain mode allows explicit knowledge storage with semantic retrieval.

RAG — Document Intelligence

Upload PDFs, notes, and documents for contextual Q&A, semantic search, and insight generation. Built on a vector retrieval pipeline with ChromaDB embeddings and context-aware injection into the AI's reasoning context.

Autonomous AI Agents

A multi-agent orchestration system with specialized roles: Coding Agent, Research Agent, Automation Agent, Memory Agent, and Scheduling Agent. A planning engine decomposes complex goals into ordered, executable steps. Agents coordinate through a shared context store and can run extended workflows in the background with live progress visibility and user cancellation at any checkpoint.

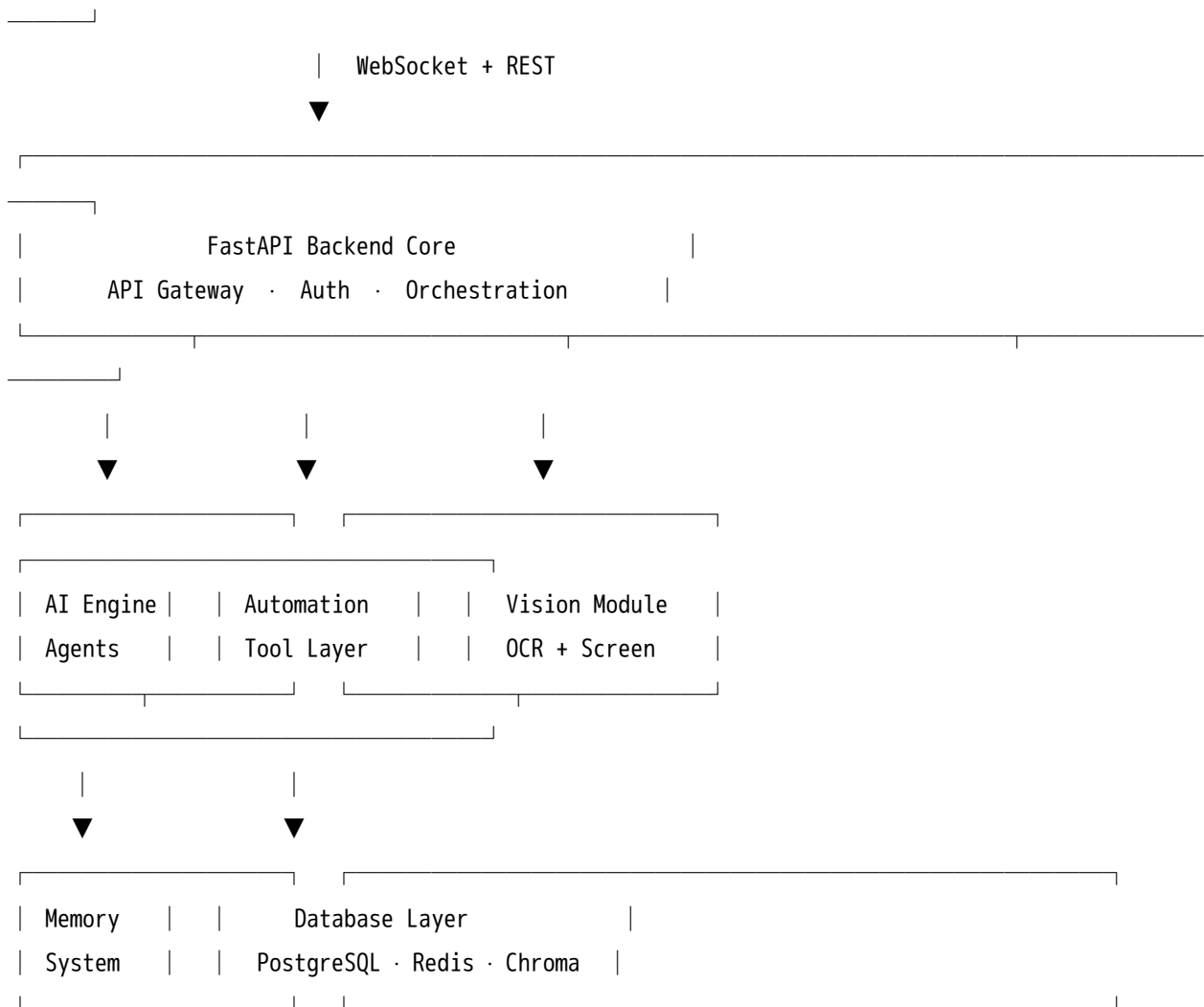
Plugin System

A Python SDK enabling third-party developers to extend GEWN with new commands, agents, workflows, and API integrations. Plugins are sandboxed, permission-scoped, and distributed through a plugin registry. Core integrations include GitHub, Notion, Gmail, Slack, Discord, Google Calendar, and Linear.

Architecture

GEWN follows a layered, modular architecture designed for realtime responsiveness, extensibility, and phased scalability.





Frontend Layer — Electron + React floating overlay providing the chat interface, voice visualization, overlay rendering, and realtime streaming UI.

API Gateway — FastAPI handles authentication, request routing, WebSocket lifecycle management, and response streaming to the frontend.

AI Engine — The reasoning core: conversational engine, multi-agent orchestration via LangGraph, planning engine for multi-step task decomposition, and tool-calling integration via LangChain.

Automation Layer — PyAutoGUI for OS-level desktop control, Playwright for browser automation, and a workflow execution engine with risk classification and rollback support.

Vision Module — Screenshot pipeline, OCR extraction (PaddleOCR primary, Tesseract fallback), multimodal LLM analysis, and Electron's transparent window layer for overlay rendering.

Voice Pipeline — Porcupine wake word → Whisper/Deepgram STT → LLM → ElevenLabs/Coqui TTS, connected by a WebSocket audio streaming layer with end-of-speech detection.

Memory System — Redis for session state, PostgreSQL for structured persistent data, ChromaDB for vector embeddings and semantic retrieval.

Processing Model — Privacy-sensitive tasks (file operations, OCR, local cache) run locally for low latency and privacy. LLM inference, advanced reasoning, and large-scale embedding run in cloud with a Privacy Mode fallback to local models via Ollama.

Tech Stack

Layer	Technology
Desktop Framework	Electron
UI Framework	React + Tailwind CSS + Framer Motion
Backend	Python + FastAPI + WebSockets
AI/LLM	OpenAI API, Google Gemini API
Agent Orchestration	LangChain, LangGraph
Wake Word	Porcupine
Speech-to-Text	Whisper (local), Deepgram (cloud)
Text-to-Speech	ElevenLabs (cloud), Coqui TTS (local)
Desktop Automation	PyAutoGUI
Browser Automation	Playwright
OCR	PaddleOCR, Tesseract
Vector Database	ChromaDB, Pinecone
Relational Database	PostgreSQL
Cache / Session	Redis
Local LLM (Privacy Mode)	Ollama
DevOps	Docker, GitHub Actions

AI Behavior & Safety

GEWN's AI behavior is governed by a formal internal specification (AI_BEHAVIOR.md) that defines how the system reasons, communicates, acts, and refuses. Every agent and subsystem within GEWN operates under these rules.

Core Behavioral Principles

Capable and direct — GEWN leads with answers and actions. It does not over-explain, hedge unnecessarily, or use performative filler. It is honest about uncertainty and never fabricates information.

Permission-first autonomy — GEWN operates with bounded autonomy. All actions are classified into four risk tiers:

Tier	Examples	Handling
T1 — Low	Read files, open apps, summarize content	Execute silently
T2 — Medium	Create files, rename, move	Confirm once
T3 — High	Delete files, run terminal commands	Explicit approval every time
T4 — Critical	Modify system settings, send communications	Typed confirmation + full action preview

Scope enforcement — GEWN does not expand beyond what the user requested. If it identifies an opportunity outside the task scope, it surfaces the observation and waits for authorization before acting.

Memory transparency — Users can ask what GEWN remembers at any time and receive a complete, accurate answer. All stored memories are viewable, editable, and deletable. GEWN never stores sensitive data (credentials, passwords, private keys) in the memory system.

Honest failure handling — When a tool fails or a workflow partially completes, GEWN reports clearly what succeeded, what failed, and what options are available. It does not retry T3/T4 actions automatically.

Hallucination prevention — GEWN never generates factual claims about the user's files, system state, or external services without reading them directly. Uncertainty is surfaced transparently, not filled with plausible-sounding fabrication.

Security Model

GEWN's security architecture recognizes that a system with screen access, file system control, terminal execution capability, and AI reasoning is a high-value target — and designs accordingly. Full documentation is in `SECURITY_NOTES.md`.

Key Security Principles

Principle of Least Privilege — Every module and integration starts with zero permissions. Access is granted only for the specific resource and duration required.

Prompt Injection Defense — External content (documents, web pages, OCR output, clipboard) is always injected into the AI context as clearly marked `[EXTERNAL_CONTENT]` data, architecturally separated from system instructions. Output validation checks that inferred actions are consistent with the original user request before execution.

File System Safety — All file paths are canonicalized and validated against an authorized scope allowlist before any operation. Path traversal attacks via `../`, symlinks, and encoded separators are blocked by design. Sensitive file patterns (private keys, `.env` files, SSH credentials, browser credential stores) are blocked from all operations.

Command Execution Safety — Terminal commands are never constructed from string interpolation of user input. Arguments are always passed as discrete subprocess parameters. An allowlist governs which command categories GEWN can invoke without elevated confirmation.

API Key Security — All secrets and credentials are stored in OS-native secure storage (macOS Keychain, Windows Credential Manager, Linux Secret Service). Keys are never written to application files, logs, or the memory system. All API communication uses TLS 1.3.

Local Encryption — Long-term memory, conversation history, and user data are encrypted at rest using AES-256-GCM. The encryption master key is stored in the OS secure enclave and never written to disk in plaintext.

Plugin Sandboxing — Third-party plugins execute in isolated subprocesses with declared permission scopes, resource limits (25% CPU, 512MB memory), and no direct access to GEWN's

internal memory or the broader file system outside a plugin-specific directory. Plugins are code-signed against the GEWN plugin registry.

Privacy Mode — When activated, all processing routes exclusively to local models and local storage. No data is transmitted to any external service. A persistent visual indicator confirms Privacy Mode is active.

Audit Log — A tamper-evident, append-only log records all consequential actions (file operations, terminal commands, API calls, permission grants) with timestamps and outcomes. The log is owned by the user and exportable at any time.

User Interaction Model

Users interact with GEWN through three entry points that all funnel into the same AI engine:

- **Voice** — "Hey GEWN" wake word, or hold-to-talk for hands-free interaction
- **Chat** — Type into the floating overlay panel for detailed requests, code help, and research
- **Keyboard shortcut** — Instant focus without switching applications

Interaction Modes

Mode	Best For
Standard	General conversation and assistance
Focus	Active deep work — GEWN minimizes interruptions
Research	Document analysis, RAG, structured output
Automation	Workflow execution with step-by-step confirmation
Voice	Real-time continuous conversation

Permission Approval UX

GEWN's permission system is designed to be safe without being obstructive. Safe, reversible actions execute silently. Moderate actions show a brief dismissable toast with an undo option. High-risk actions surface a blocking confirmation dialog showing exactly which files or commands are affected. Critical actions require typed confirmation.

All executed workflows are logged, and reversible actions offer an undo window within the session.

Roadmap

GEWN is built in seven deliberate phases — each independently shippable, each laying the foundation for the next. No phase begins until the previous one is stable.

Phase	Focus	Timeline Estimate
Phase 1 — MVP Core	Floating UI, conversational AI, memory, basic file/app control, RAG	10–16 weeks
Phase 2 — Voice AI	Wake word, STT/TTS pipeline, real-time voice mode	8–12 weeks
Phase 3 — Desktop Automation	Advanced file management, browser automation, workflow presets, scheduler	10–14 weeks
Phase 4 — Vision & Context	OCR, screen understanding, coding error detection, visual overlays, proactive assistance	10–14 weeks
Phase 5 — Autonomous Agents	Multi-agent orchestration, planning engine, background task execution	12–16 weeks
Phase 6 — Plugin Ecosystem	Developer SDK, core integrations (GitHub, Notion, Gmail, Slack), plugin registry	12–16 weeks
Phase 7 — Multi-Device	Cross-device sync, mobile companion app, local AI (Ollama), enterprise workspace	14–20 weeks

The guiding principle across all phases: **ship something real before adding something ambitious.**

Project Structure

```
project-gewn/  
|  
|—— frontend/  
|   |—— electron/           # Desktop shell, window management, IPC  
|   |—— src/  
|   |—— components/        # Chat, voice UI, overlay, permission dialogs  
|   |—— hooks/              # WebSocket, streaming, media hooks
```

```
|   |—— animations/      # Framer Motion animation definitions
|   |—— services/        # Frontend API clients
|
|—— backend/
|   |—— app/             # FastAPI application core
|   |—— routes/          # API endpoint definitions
|   |—— agents/          # AI agent implementations (coding, research, automation, memory)
|   |—— rag/             # Document ingestion, embedding, retrieval pipeline
|   |—— memory/          # Short-term, long-term, and semantic memory management
|   |—— automation/      # File system tools, app control, workflow engine
|   |—— vision/          # OCR pipeline, screenshot processing, overlay engine
|   |—— websocket/       # WebSocket server and streaming layer
|   |—— database/        # Database models and migration management
|
|—— ai-models/           # Local model configurations and prompt templates
|—— docs/                # Architecture and specification documents
|—— docker/              # Docker Compose and environment configs
|—— scripts/             # Development and deployment utility scripts
|—— README.md
```

Documentation

All specification documents are in the `/docs` directory:

Document	Description
PRD.md	Full product requirements document — goals, user scenarios, functional and non-functional requirements
ARCHITECTURE.md	System architecture specification — all layers, components, data flows, and technology rationale
FEATURES.md	Detailed feature specifications with implementation guidance for each capability
ROADMAP.md	Phased development plan with milestones, technical goals, expected challenges, and a risk register
AI_BEHAVIOR.md	Internal AI behavioral specification — personality, safety rules, permission handling, memory behavior, and ethical guidelines governing all AI components
SECURITY_NOTES.md	Security architecture — threat model, permission system, file/command safety, encryption, privacy, and prompt injection defenses

Document	Description
USER_FLOW.md	Comprehensive user flow diagrams for all major interaction patterns — voice, chat, automation, file management, memory, vision, and permissions

Development Status

Current phase: Active architecture and planning. Development environment configuration in progress.

Next milestone: M1.1 — Electron overlay shell: GEWN window launches, floats on desktop, can be dragged and minimized.

Contributions, architecture discussions, and feature proposals are welcome. Contribution guidelines, branching strategy, and issue templates will be published alongside the first working milestone.

GEWN is not designed as a simple chatbot. It is designed as an intelligent operating companion capable of evolving into a fully AI-native computing environment.